练习 1.7（面向对象 + matplotlib 版）

对应内容：教材 (1.1) 式 $dN_U/dt = -N_U/\tau$、(1.2) 式解析解、(1.3)–(1.4) 式 Euler 格式、以及 1.5 节的截断误差与稳定性讨论。
配合课程：《Python 面向对象与函数式编程》单元 6（`@property` / `@classmethod`）与单元 9（装饰器）。
建议用时：90–120 分钟。**难度**：★★☆（比教材 1.1–1.3 多一层工程要求，物理内容不变）。

一、题目背景

教材里我们已经用 Euler 法算过放射性衰变，并发现"缩小时间步长 Δt 会让数值解逼近解析解"。现在请你把这套计算**封装成一个类**，并画出**一张能自己说明问题的图**——不仅是把曲线画出来，而是要让图上的证据回答：

1. Euler 法的误差是随机的还是系统性的?
2. 误差随 Δt 如何变化？这能否证明 Euler 法是"一阶方法"?
3. Δt 大到什么程度，数值解会彻底失效?

本题**必须**使用 `@property`（至少要有一个带 setter 的校验属性、一个"算出来"的计算属性），并用 `matplotlib` 出图。

二、程序要求

写一个模块 `decay_oop.py`，实现类 `Decay`，对外接口**必须**与下表一致（评分按此接口自动核对）：

接口	类型	含义
`Decay(N0, tau, dt, t_max)`	构造	初值、时间常数、步长、模拟总时长
`d.t`	`@property` → `ndarray`	与 Euler 取值点严格对齐的网格 $t_k = k\Delta t$
`d.numerical`	`@property` → `ndarray`	Euler 数值解 (1.4)
`d.exact`	`@property` → `ndarray`	解析解 (1.2) $N_0 e^{-t/\tau}$，取样点同 `d.t`
`d.error`	`@property` → `ndarray`	**带符号**的逐点误差 `numerical - exact`

接口	类型	含义
`d.max_abs_error`	`@property` → `float`	最大绝对误差
`d.half_life`	`@property` → `float`	半衰期 $\tau \ln 2$
`d.N0` / `d.tau` / `d.dt`	`@property` + `@setter`	读得到；写入非法值（≤ 0）时抛 `ValueError`
`Decay.euler_step(n, dt, tau)`	`@staticmethod`	单步 Euler：$n - n\,\Delta t/\tau$
`Decay.from_half_life(N0, t_half, dt, t_max)`	`@classmethod`	用半衰期构造：$\tau = t_{1/2}/\ln 2$

实现要点：

- **计算属性必须"现算"，不允许缓存。** 这样 `d.dt = 0.05` 之后 `d.max_abs_error` 自动更新，不会读到过期结果。
- 用 `@setter` 做校验：`Decay(100, -1.0, 0.2, 5)` 与 `d.dt = 0` 都必须抛 `ValueError`。
- 浮点数比较用 `np.allclose`，不要在循环里拼接 Python list 再转 `ndarray`（可以，但要说明理由）。

三、绘图要求

用 `matplotlib` 画一张 **1×3 子图**（`figsize=(15, 4.2)`），保存为 `decay_oop.png`（dpi ≥ 140）：

面板	内容	必须有的标注
左	解析解曲线 + Euler 数值解散点（$\Delta t = 0.2$ s）	横轴"时间 t/s"、纵轴"剩余核数 N"、图例区分两条线
中	逐点误差 $N_{\text{num}} - N_{\text{exact}}$ 随 t 的变化	零参考线；标题里给出最大绝对误差的数值
右	**log-log** 图：最大绝对误差 vs Δt（$\Delta t = 0.5, 0.25, 0.2, 0.1, 0.05, 0.025$）	一条斜率为 1 的参考虚线；标题给出 `np.polyfit` 拟合的斜率

参数固定为 $N_0 = 100$、$\tau = 1$ s、$t \in [0, 5]$ s。

图面质量要求（教材 1.5 节"程序编写指南"的同一标准）：坐标轴必须有量与单位；中文标签不能出现方框（需设置 `font.sans-serif`，见提示 3）；log 轴刻度不要挤成一团。

四、分析题（写在代码文件的 docstring 或单独的 `README` 里，不超过 10 行）

1. `error` 为什么**恒为负**？从 $(1-\Delta t/\tau)^n$ 与 $e^{-t/\tau}$ 的大小关系说明。
2. 右图斜率 ≈ 1 说明了什么？如果把 Euler 换成"每步用 $N_{k+1}=N_k e^{-\Delta t/\tau}$"（精确递推），右图会变成什么样？
3. 取 $\Delta t = 2.5$ s（即 $> 2\tau$）重跑，数值解发生了什么？用 $|1-\Delta t/\tau|$ 的取值解释这个"稳定性边界"。（教材 1.5 节讨论过稳定性，本题要求你**用实验确认它**。）

五、自查表（提交前逐项核对）

用 $N_0 = 100$、$\tau = 1$ s、$t_{\max} = 5$ s 运行，结果应落在下表范围内（容差 ±0.5%）：

检查项	期望值
半衰期 `d.half_life`	0.6931 s
$\Delta t = 0.2$ 时 `N(5)`	0.3778（解析值 0.6738）
$\Delta t = 0.2$ 时 `max_abs_error`	4.0199（出现在 $t = 1$ s 附近）
$\Delta t = 0.05$ 时 `max_abs_error`	0.9394
误差比（Δt 减半时的误差比）	趋于 2.0（$0.5 \to 0.25$ 一格为 2.29，$0.05 \to 0.025$ 已到 2.02）
log-log 拟合斜率	1.0 ± 0.1
$\Delta t = 2.5$、$t = 25$ 时的 $\lvert N\rvert$	发散（量级 10^3 以上）

另外：

- [ ] `Decay(100, -1.0, 0.2, 5)` 抛 `ValueError`
- [ ] `d.dt = 0.05` 后 `d.max_abs_error` 立刻从 4.0199 变为 0.9394（证明属性现算、无过期缓存）
- [ ] `Decay.from_half_life(100, 0.6931, 0.2, 5)` 与 `Decay(100, 1.0, 0.2, 5)` 的 `max_abs_error` 一致（实测相对差 7×10^{-5}，差异只来自 0.6931 这个四舍五入的半衰期）
- [ ] 图中无中文缺字方框、无未标注坐标轴

六、评分要点（满分 100）

项	分值	判定
接口与装饰器齐备（`@property`/setter/`@staticmethod`/`@classmethod` 各到位）	30	缺一项扣 5–8 分
计算属性现算、改参数后结果自动更新	15	出现手动"重算"调用或缓存过期即扣满
三面板图内容、标注、单位、图例	25	缺单位/图例/参考线各扣 5
分析题 3 问（符号、阶、稳定性）	20	每问 5–7 分，只看物理与数值解释
自查表通过	10	每项不符扣 2

七、提示

1. **别忘了解析解的取样点**要和 Euler 的取值点相同（`d.t`），否则误差曲线会出现虚假的锯齿。
2. `np.polyfit(np.log(dts), np.log(errs), 1)[0]` 就直接给出 log-log 斜率。
3. 中文字体（macOS/Linux 通用写法）：

```
plt.rcParams["font.sans-serif"] = ["Source Han Sans SC", "Noto Sans CJK SC",
                                   "PingFang SC", "Heiti SC",
"SimHei", "Arial Unicode MS"]
plt.rcParams["axes.unicode_minus"] = False
```

4. 画图前调用 `matplotlib.use("Agg")`，在没有显示器的环境（服务器、CI）里也能存图。